RV College of Engineering®

# AI Foundations for Engineers
# CI124TA
# UNIT 3


Nithyashree G D

Assistant Professor,

Department of Computer Science and Engineering,

RVCE

RV College of Engineering®

# Outline

- **Introduction to Machine Learning**
- **Decision Tree Learning**
- **Bayesian Learning**
- **Instance-Based Learning**

RV College of Engineering®

# Early History & Progress of Machine learning

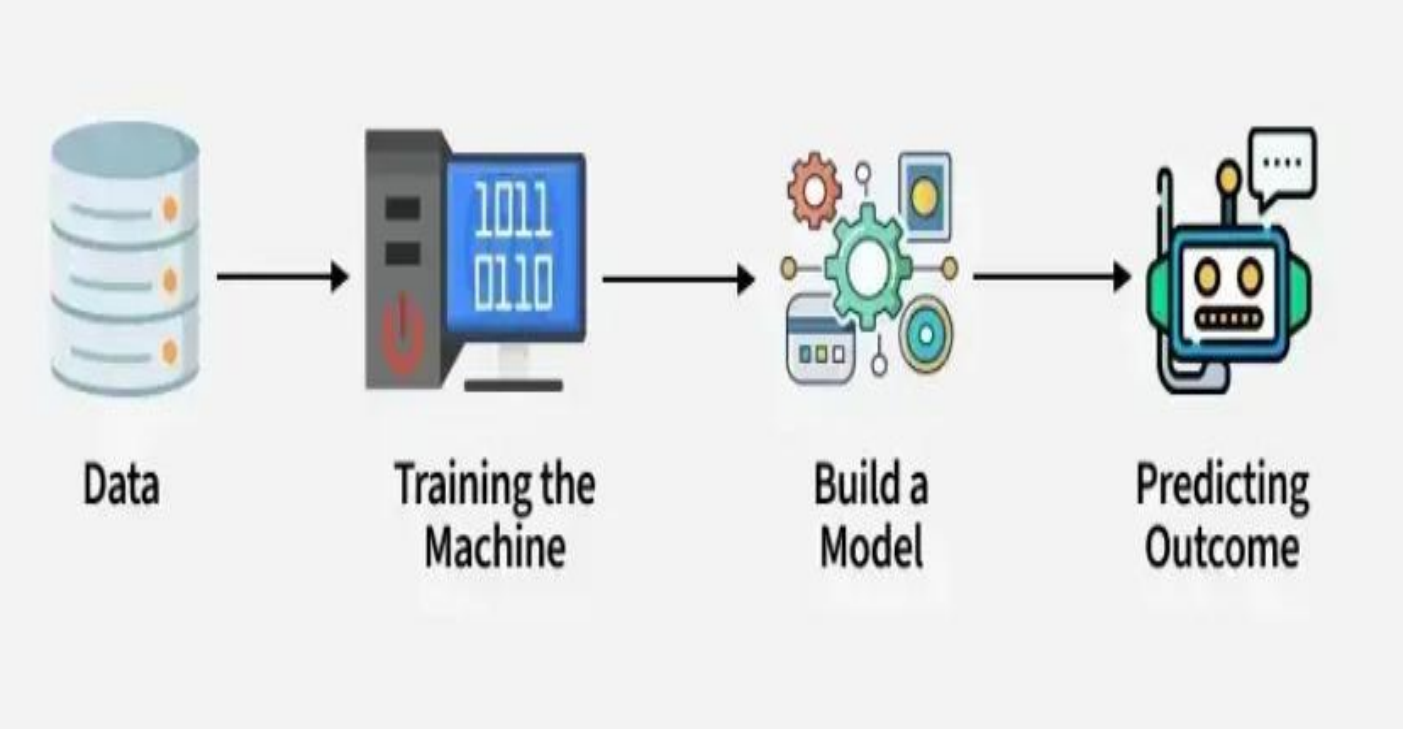
- Since computers were invented, researchers have asked: Can machines learn?
- Machines still cannot learn as well as humans- but progress is significant.
- Effective algorithms now exist for specific learning tasks.

Potential uses:

- Medical diagnosis & treatment recommendations
- Smart homes optimizing energy use
- Personal assistants adapting to user interests

# What is Machine Learning?

- Machine Learning means teaching computers to learn from examples (data) and make predictions.



RV College of Engineering

# Well Posed Learning Problem

- **A computer program is said to learn from experience E with respect to some class of tasks *T* and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.**

## Clearly define three things:

- **Task (T)** — What the program should do.

- **Experience (E)** — How it learns (data, examples, practice).
- **Performance Measure (P)** — How you measure if it's getting better.

RV College of Engineering

# Well Posed Learning Problem

## Example — Email Spam Filter

**Task (T)**: Classify emails as spam or not spam.

**Experience (E)**: The program learns from examples of emails labeled "spam" or "not spam."

**Performance Measure (P)**: The percentage of emails correctly classified (accuracy).

## Example : Robot Driving Learning Problem:

**Task :** driving on public four-lane highways using vision sensors.

**Performance measure :** average distance traveled before an error (as judged by human overseer)

**Experience :** a sequence of images and steering commands recorded while observing a human driver

RV College of Engineering

# Designing a Learning System

## Choosing the Training Experience:

- Where will the learner get its experience or data? Will it get clear feedback or figure things out on its own?

## Choosing the Target Function:

- What exactly do we want the learner to predict or decide? This is the goal or “rule” we want it to discover.

## Choosing a Representation for the Target Function:

- How will the learner store or express what it has learned? (e.g., decision trees, rules, neural networks, etc.)

## Choosing a Function Approximation Algorithm:

- What learning method or algorithm will we use to find the target function from the training examples?

RV College of Engineering®

# Designing a Learning System -Example

- **Teaching a Child About Fruits**
- **Well-posed learning problem**
  - Problem: “Given a fruit, tell me its name.”
  - Input: A fruit shown to the child.
  - Output: The correct fruit name.
  - Success measure: How often the child gets it right.
- **Choosing the training experience**
  - You show the child many fruits: apples, bananas, grapes.
  - Each fruit shown is a *training example*.
- **Choosing the target function**
- The ideal skill: $f\,(fruit) \rightarrow name\ of\ fruit$.
- Example: If shown a banana, the child should say “banana.”

RV College of Engineering®

# Designing a Learning System

**Choosing representation for the target function**

- How does the child "store" this knowledge?
- Maybe by remembering color, shape, or taste.
- Example: "Bananas are long and yellow," "Apples are round and red."

**Choosing a function approximation algorithm**

- This is the *learning method*.
- The child might learn by repetition, flashcards, or tasting.
- Example: You quiz them daily until they get it right.

RV College of Engineering®

# Designing a Learning System

- Teaching a Computer to Recognize Dogs in Photos
- Well-posed learning problem
- Choosing the training experience
- Choosing the target function
- Choosing representation for the target function
- Choosing a function approximation algorithm

# Choosing the Training Experience

## Direct vs Indirect Feedback

**Direct Feedback:**

- The program gets clear, exact guidance.
- *Example:* Someone tells the program the **best move** for each board position.
- *Easy to learn — the program knows exactly what's right or wrong.*

**Indirect Feedback:**

- The program only sees whether it **won or lost** the whole game.
- *It has to figure out **which moves were good or bad** by itself.*
- *Harder — because sometimes a good move might still lead to losing if later moves are bad.*

RV College of Engineering

# Choosing the Training Experience

## Who Controls the Training Examples?.

**Teacher-controlled:** A human teacher shows important situations and the best moves.

**Learner-controlled (Self-play)**

## Does the Training Look Like Real-World Testing?

**Choosing the right training experience means:**

- Give clear feedback if possible.
- Make sure the learner sees examples similar to what it will see in the real world.
- Decide who controls what it learns from — a teacher, the learner itself, or both.

RV College of Engineering

# Choosing the Target Function

When you design a learning system, you must decide: What exactly do you want the program to learn to do? You want to build a computer program that predicts how much a house will sell for.

**Task (T):** Predict house price

**Experience (E):** Past data about houses — size, location, number of bedrooms, age, and the actual price they sold for.

**Performance (P):** How close the predicted price is to the real selling price.

## Direct Way: Predict the Exact Price

The obvious goal is:

**Input:** Details about the house

**Output:** Predicted price in dollars

**Target function** is:

**Price: H→R** where H is the set of all possible houses (with features like size, location, etc.), and R means it outputs a real number (the price).

RV College of Engineering

# Choosing the Target Function

## But what if direct prediction is too complex?

- Suppose the housing market is huge and pricesdepend on complex factors- neighborhood trends, school ratings, upcoming development projects — some of which are hard to model directly.
- In that case, you might simplify the target function: Instead of predicting the exact price
- Predict a score: how desirable is the house?

**Desirability→[0,100]**

# Choosing a representation for the Target Function

- The computer does not store knowledge the way humans do.
- It needs a clear, formal way to store, modify, and use what it has learned.

**The representation you choose decides:**

1. How easy it is for the learner to learn it.
2. How well it works for real problems.
3. How fast predictions can be made.
4. Whether humans can understand it!

# Choosing a representation for the Target Function

**Spam Email Classifier**

Target function: Is this email spam or not?

$f: E \rightarrow \{spam, not\ spam\}$

**Set of Rules:**

"IF email contains 'win lottery' AND has an unknown sender THEN mark as spam."

## Decision Tree:

A tree where you ask questions:

Does the subject contain 'Congratulations'?

Does the sender have a trusted domain?

What is the probability of spam?

## Linear Model:

A mathematical formula like:

Spam Score=2.3×(Has 'free')+1.1×(Contains link)−0.5×(Sender known)

RV College of Engineering®

# Choosing a Function Approximation Algorithm

Decide **what you want to learn** (the target function)

Decide **how you'll represent it** (rules, decision tree, neural network, etc.)

…the next big question is:

**HOW will you actually *learn* it from the data?**

This is where the **Function Approximation Algorithm** comes in.

A **Function Approximation Algorithm** is the *learning method* that adjusts your chosen representation to fit the training data.

- If your representation is a decision tree → you need a tree-learning algorithm.
- If your representation is a neural network → you need an algorithm that adjusts the network's weights.
- If it's a line or curve → you need an algorithm that finds the best line or curve

RV College of Engineering®

# Adjusting the Weights

$$E = \sum (true - predicted)^2$$

For each training example, you know:

- The true value (from training data)
- The predicted value (from your model)

RV College of Engineering®

# Task

- Build Your Own Learning System
  - **Step 1: Pick a Problem**
  - Everyday examples: choosing transport, recommending movies, classifying fruits.
  - **Step 2: Define Inputs (Features) and outputs(Decisions)**
  - What information will the system use?
  - What decision should the system make?
  - **Step 3: Choose the Target function**
  - Define the goal mapping from input -> output
  - Ex: f(study hours) -> exam score
  - **Step 4: Choose Representation**
  - How will the decision be stored?
  - Options: decision tree, rule list, linear equation, neural network.

RV College of Engineering®

# Task

- **Step 5: Choose Learning Algorithm**
- How will the system learn?
- Examples: information gain (trees), gradient descent (networks), least squares (regression).
- **Step 6: Present the Final Design**
- Summarize: problem, inputs, outputs, target function, representation, algorithm.
- Show how the system makes a decision with sample data.

RV College of Engineering®

# Ex- Checkers Learning Problem

- **Task**
- **T:** Playing checkers
- **Performance Measure**
- **P:** Percent of games won in world tournament
- **Training Experience**
- **E:** Games played against itself (self-play generates unlimited data)

- **Target Function**
- **V(b):** Evaluation function assigning a score to board state *b*
- Win → +100
- Loss → –100
- Draw → 0
- Otherwise → value of best achievable final state

RV College of Engineering®

# Ex- Checkers Learning Problem

- **Representation of Target Function**
- Linear combination of board features:
  - $x_1$: number of black pieces on the board
  - $x_2$: number of red pieces on the board
  - $x_3$: number of black kings on the board
  - $x_4$: number of red kings on the board
  - $x_5$: number of black pieces threatened by red
  - $x_6$: number of red pieces threatened by black
- Our learning program will represent V(b) as a linear function of the form
- $V(b) = w_0 + w_1x_1 + w_2x_2 + \cdots + w_6x_6$

where $w_0$ through $w_6$ are numerical coefficients, or weights, to be chosen by the learning algorithm.

The learned weights $w_1$ through $w_6$ show how important each board feature is, while $w_0$ acts as a constant baseline value for the board.

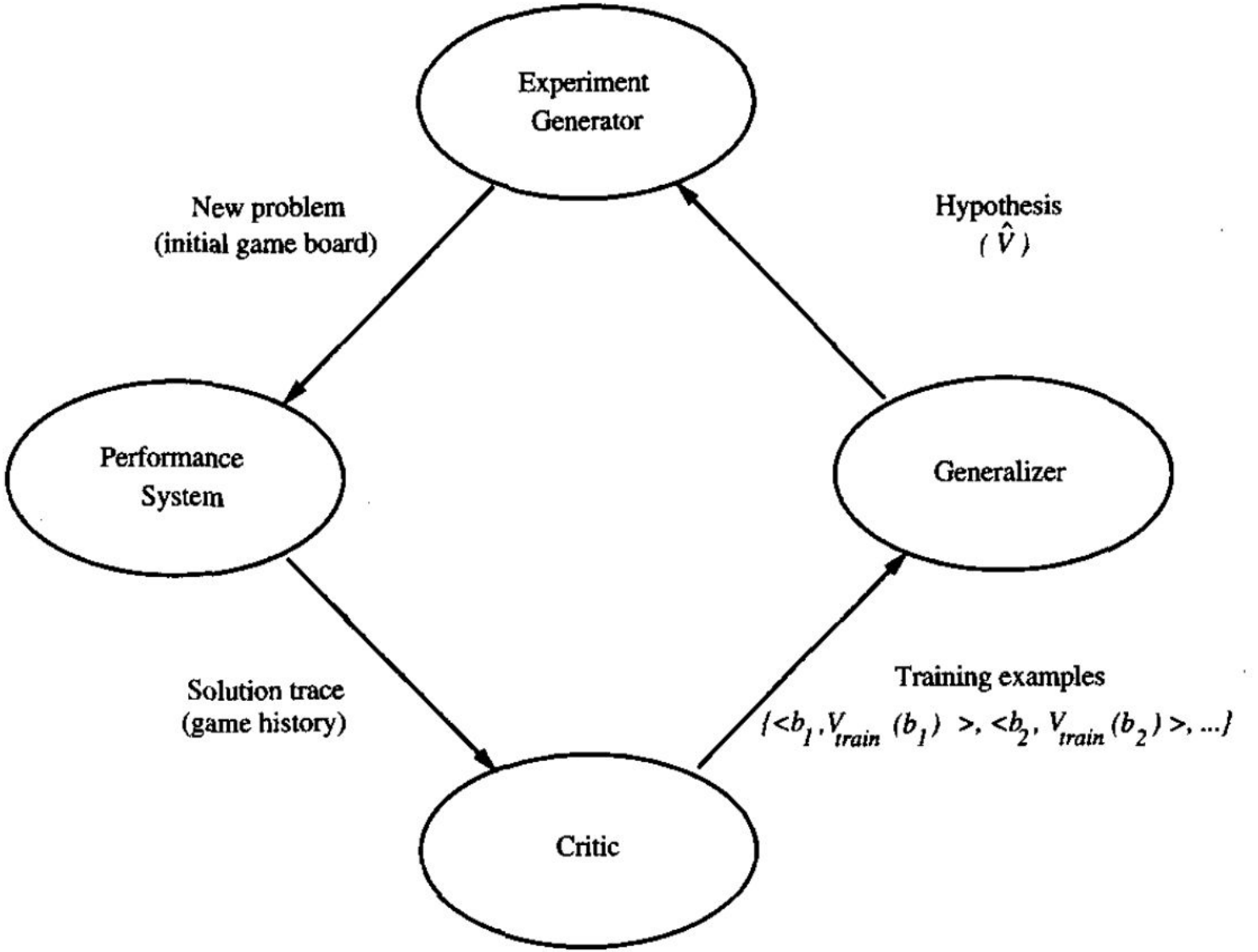
RV College of Engineering®

# Ex- Checkers Learning Problem

- **Function Approximation Algorithm**
- The system doesn't know the right "importance numbers" at first.
- It plays games, sees results (win/loss), and **adjusts the numbers** to improve.
- Example analogy:
- If the system notices that having more kings often leads to wins, it increases the importance of "number of kings."
- If having many threatened pieces often leads to losses, it decreases that importance.
- Over time, the formula becomes better at predicting which moves are good.

RV College of Engineering®

# Final Design- Checkers Problem



RV College of Engineering®

# Final Design System

**1. Performance System**

- Plays the game using the learned evaluation function.
- Input: new board state → Output: game history (moves played).
- Improves as the evaluation function becomes more accurate.

**2. Critic**

- Looks at the game history.
- Produces **training examples** (board states + estimated scores).
- Helps the system learn which positions are good or bad.

**3. Generalizer**

- Takes training examples and builds a **general rule/function**.
- Learns patterns beyond specific examples.
- In checkers: learns a formula with weights for board features.

**4. Experiment Generator**

- Chooses new practice problems (board states).
- Ensures the system keeps learning efficiently.
- In checkers: starts new games from the same initial board.

RV College of Engineering®

# Issues in ML

**1. Algorithms**

- Which algorithms can learn general functions from examples?
- When do they converge to the right solution?
- Which work best for different problems?

**2. Training Data**

- How much data is enough?
- How does data size affect confidence in results?

**3. Prior Knowledge**

- Can existing knowledge guide learning?
- Is it useful even if partly wrong?

**4. Choosing Training Experience**

- What's the best way to pick the next example?
- How does this choice affect learning speed and complexity?

**5. Function Approximation**

- How to reduce tasks into functions to learn?
- Which specific functions should be targeted?
- Can this process be automated?

**6. Representation**

- How can the learner change/improve its representation?
- Better representation → better learning.

RV College of Engineering

# How ML works

1. Collect Data: Gather examples (numbers, text, images, etc.)
2. Train Model: Show examples to the computer to learn patterns.
3. Test Model: Check learning using new examples.
4. Use Model: Apply it to real-life predictions.

Example: Like teaching a child: show many apples and bananas, test with a new fruit.

Input Data = The examples you give (like pictures of fruits, sounds, or text).

Labels = The correct answers for each example (like “apple” or “banana”).

Learning = The machine looks at the data again and again to find patterns.

Prediction = After learning, it tries to guess new examples by applying what it learned.

RV College of Engineering

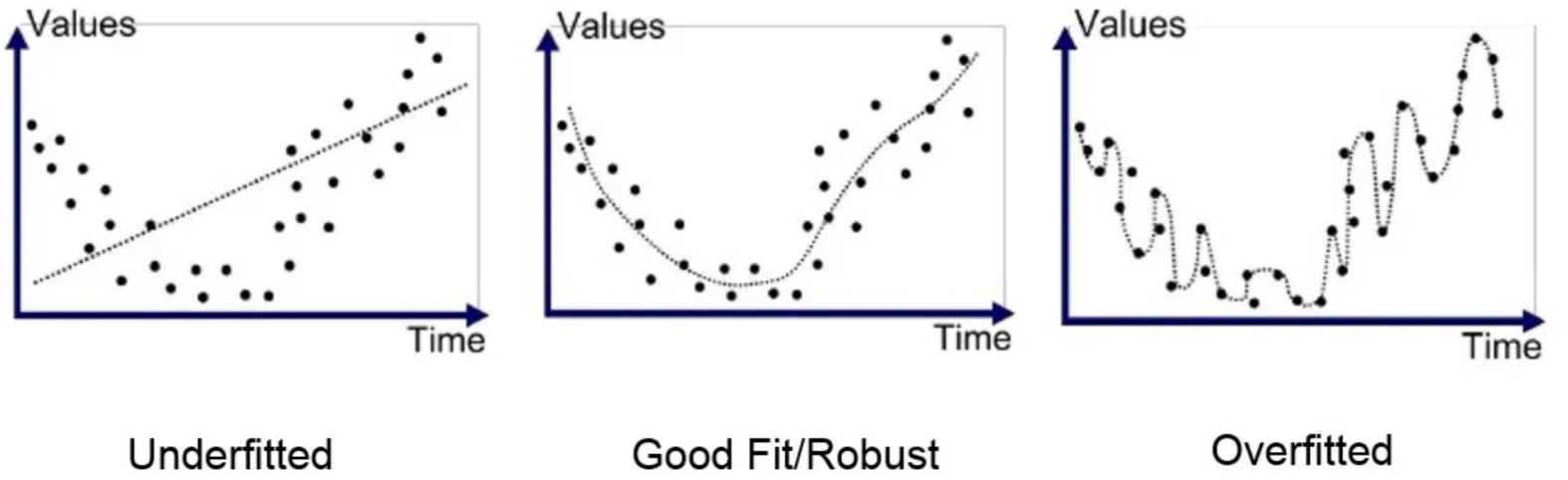
# How ML works

## Overfitting:

- A teacher teaches only from the last 5 years' question papers.
- Students do well if the same questions repeat — fail badly if even slightly new ones come.

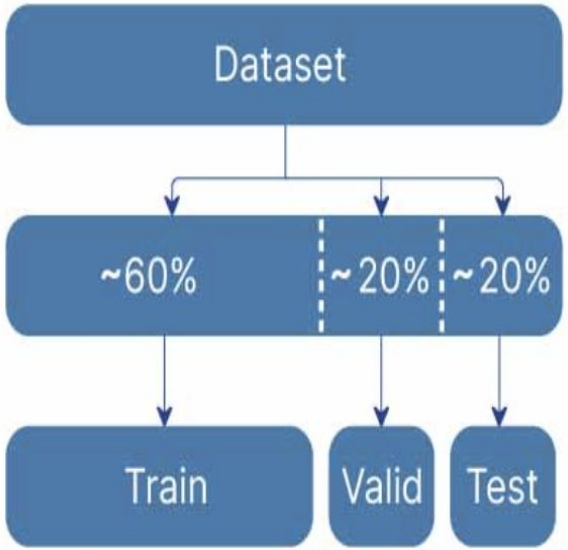
## Underfitting:

- The teacher is vague, just talks theory with no examples or practice.
- Students don't learn how to answer any questions at all.



# What is Train-Test Split

- **We divide data into two parts:**
  - **Training Set (80%): Used to teach the model.**
  - **T est Set (20%): Used to check learning.**
  - **Example: Study 8 chapters (training), exam from 2 new chapters (testing).**
- **If we train and test on same data, model may memorize answers.**
- **We want the model to understand patterns, not memorize.**
- **Example: Reading answers before exam vs truly understanding the topic.**



# Evaluation Metrics(Judging the Model)

•Accuracy: How many predictions were correct?

•Precision: Out of predicted 'YES', hc

many were actually 'YES'?

•Recall: Out of actual 'YES', how

many were caught?

• F1 Score: Balance between

Precision & Recall

## CONFUSION MATRIX

**TP** = True Positives
**TN** = True Negatives
**FP** = False Positives
**FN** = False Negatives

| | p' (Predicted) | n' (Predicted) |
|---|---|---|
| p (Actual) | True Positive | False Negative |
| n (Actual) | False Positive | True Negative |

$$\textbf{Precision} = \frac{TP}{TP + FP}$$

$$\textbf{Recall} = \frac{TP}{TP + FN}$$

$$\textbf{F1-score} = \frac{2 * precision * recall}{precision + recall}$$

# Evaluation Metrics(Judging the Model)

- Total emails = **100**
- Actual spam (YES) = **30**
- Actual not spam (NO) = **70**

Model predictions:

- Predicted spam = **40**
  - Out of these:
    - **25 are actually spam** → TP
    - **15 are not spam** → FP
- Missed spam emails = **5** → FN

**Step 1: Find TN**

Actual NO = 70

FP = 15 (wrongly marked as spam)

👉 Remaining correctly identified:

**TN = 70 – 15 = 55**

## CONFUSION MATRIX

**TP** = True Positives
**TN** = True Negatives
**FP** = False Positives
**FN** = False Negatives

| | p' (Predicted) | n' (Predicted) |
|---|---|---|
| p (Actual) | True Positive | False Negative |
| n (Actual) | False Positive | True Negative |

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1-score} = \frac{2 * \text{precision} * \text{recall}}{\text{precision} + \text{recall}}$$

RV College of Engineering

# Evaluation Metrics(Judging the Model)

| | Actual Spam | Actual Not Spam |
|---|---|---|
| Predicted Spam | 25 (TP) | 15 (FP) |
| Predicted Not Spam | 5 (FN) | 55 (TN) |

# Evaluation Metrics(Judging the Model)

**Step 2: Metrics**

🎯 **Accuracy**

Accuracy= (TP+TN) / Total

=(25+55)/100=80% Accuracy

👉 80% emails classified correctly

🎯 **Precision**

Precision=TP/(TP+FP)

=25/40=62.5%Precision

👉 Out of predicted spam, **62.5% are truly spam**

🎯 **Recall**

Recall= TP/(TP+FN)

=25/30 ≈ 83.3%Recall

👉 Model caught **83.3% of actual spam**

🎯 **F1 Score**

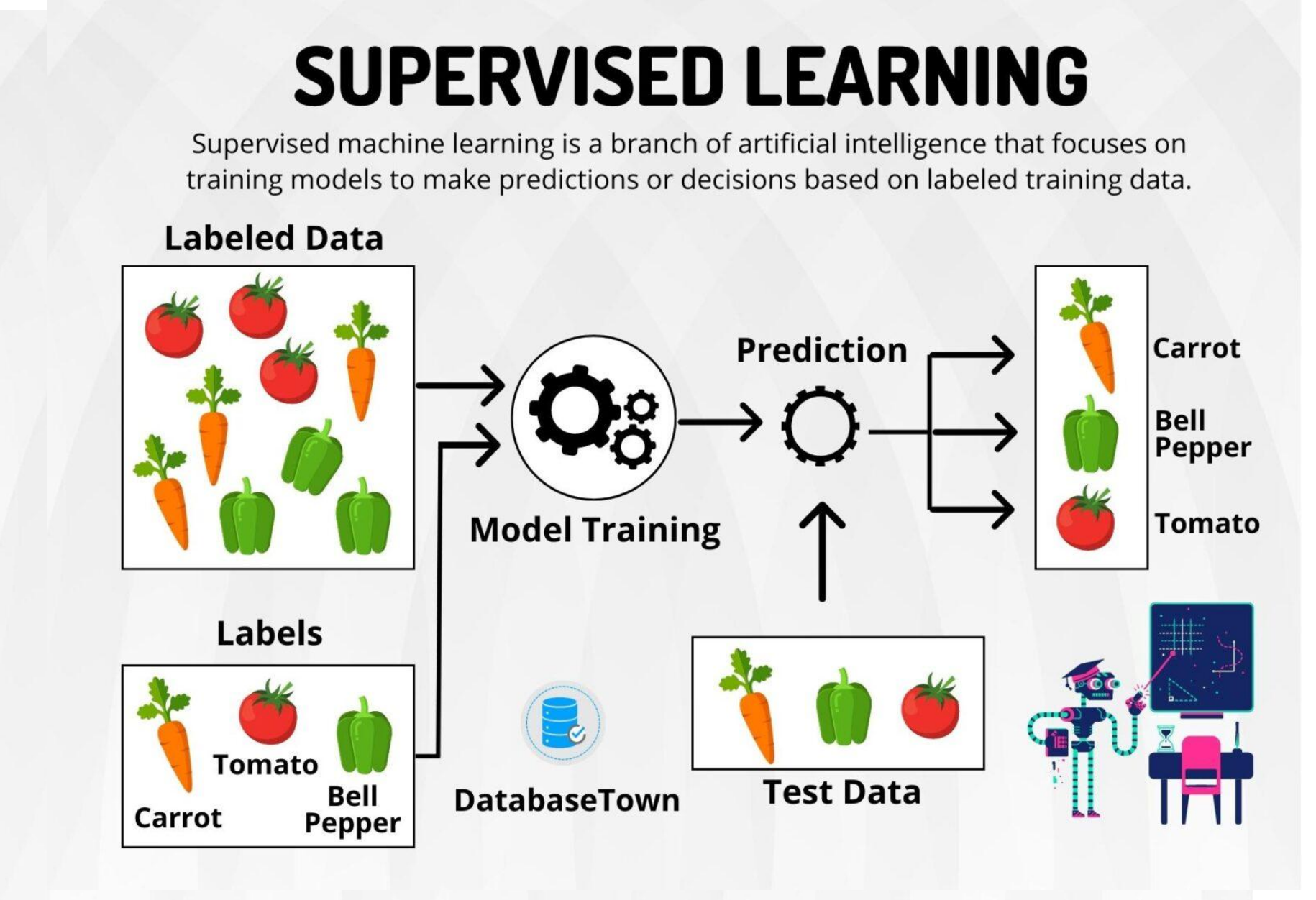
F1≈2×(0.625×0.833)/(0.625+0.833) ≈ 71.4%F1

RV College of Engineering®

# SUPERVISED LEARNING



# Introduction



RV College of Engineering®

# Introduction

- Given a collection of records (training set )
  - Each record is by characterized by a tuple ($\boldsymbol{x}$,$y$), where $\boldsymbol{x}$ is the attribute set and $y$ is the class label
    - $\boldsymbol{x}$: attribute, predictor, independent variable, input
    - $y$: class, response, dependent variable, output
- Task:
  - Learn a model that maps each attribute set $\boldsymbol{x}$ into one of the predefined class labels $y$

RV College of Engineering®

# Introduction

| Task | Attribute set, $\boldsymbol{x}$ | Class label, $y$ |
|---|---|---|
| Categorizing email messages | Features extracted from email message header and content | spam or non-spam |
| Identifying tumor cells | Features extracted from x-rays or MRI scans | malignant or benign cells |
| Cataloging galaxies | Features extracted from telescope images | Elliptical, spiral, or irregular-shaped galaxies |

# General Framework For Classification

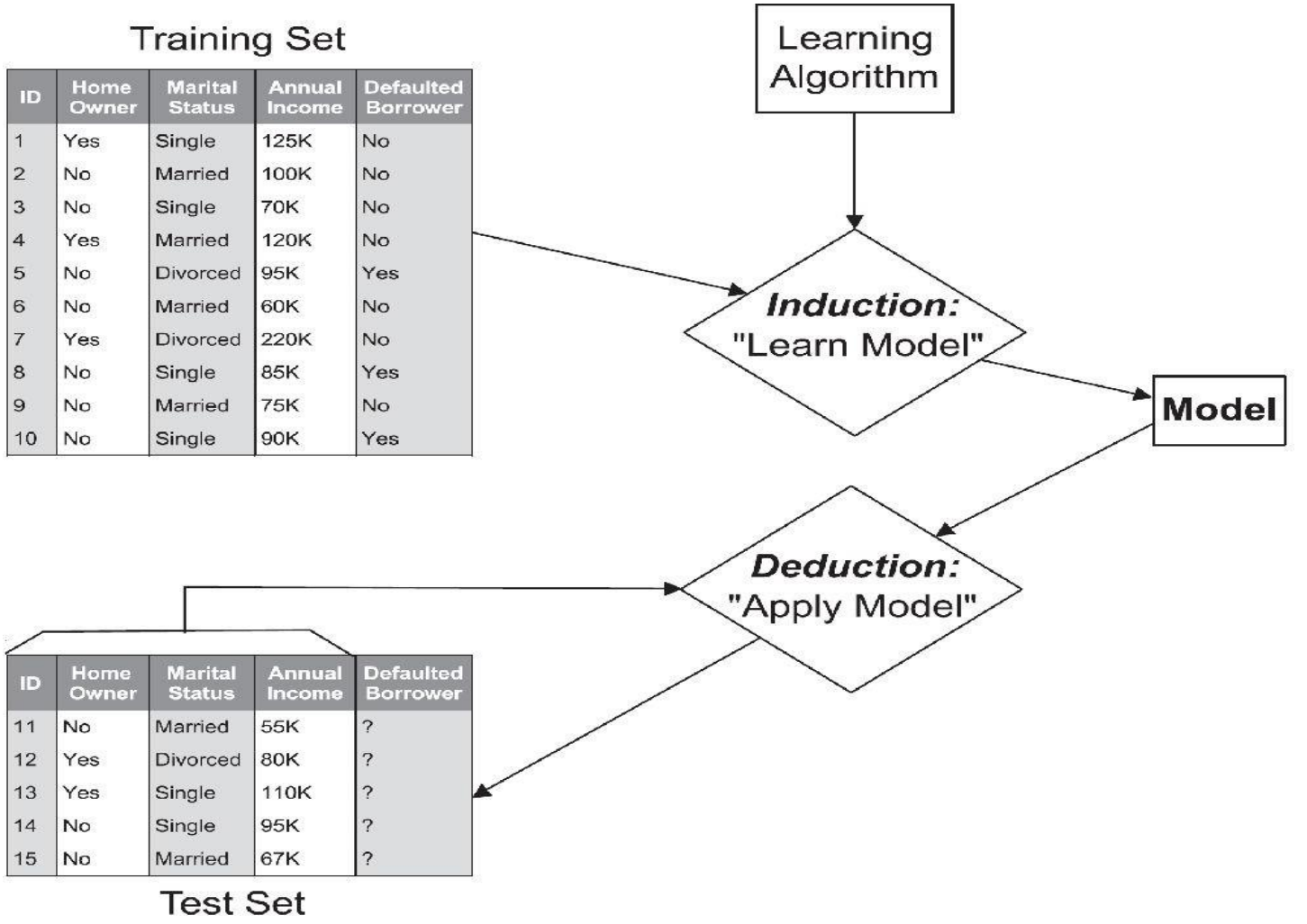



**Figure 3.3.** General framework for building a classification model.

# General Framework For Classification

**Table 3.4.** Confusion matrix for a binary classification problem.

| | | Predicted Class | |
|---|---|---|---|
| | | Class = 1 | Class = 0 |
| Actual Class | Class = 1 | $f_{11}$ | $f_{10}$ |
| | Class = 0 | $f_{01}$ | $f_{00}$ |

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}.$$

$$\text{Accuracy} = \frac{f_{11} + f_{00}}{f_{11} + f_{10} + f_{01} + f_{00}}.$$

$$\text{Error rate} = \frac{\text{Number of wrong predictions}}{\text{Total number of predictions}} = \frac{f_{10} + f_{01}}{f_{11} + f_{10} + f_{01} + f_{00}}.$$

RV College of Engineering®

# Classification Techniques

- Base Classifiers
  - **Decision Tree based Methods**
  - Rule-based Methods
  - **Nearest-neighbor**
  - **Naïve Bayes** and Bayesian Belief Networks
  - Support Vector Machines
  - Neural Networks, Deep Neural Nets

- Ensemble Classifiers
  - Boosting, Bagging, Random Forests

RV College of Engineering

# Decision Tree Introduction

- A method for approximating discrete-valued target functions.
- The learned function is represented as a decision tree.
- Trees can be converted into **if-then rules** for easier human readability.

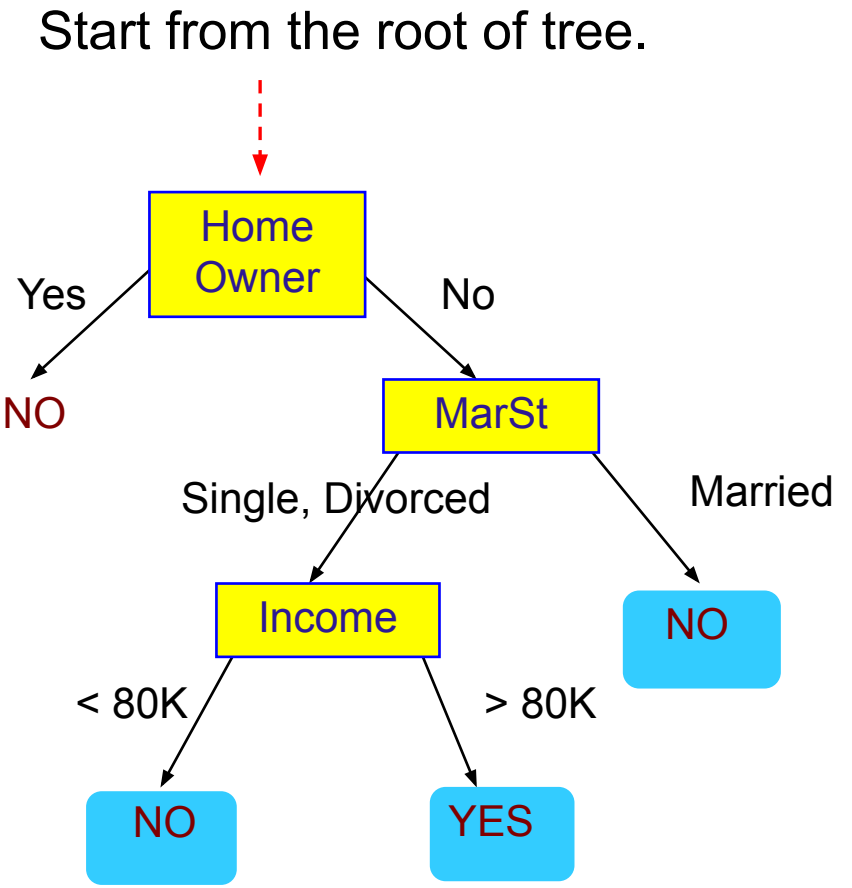
**Applications**

Diagnosing medical cases.

Assessing credit risk of loan applicants.

Many other classification tasks.

# Apply Model to Test Data



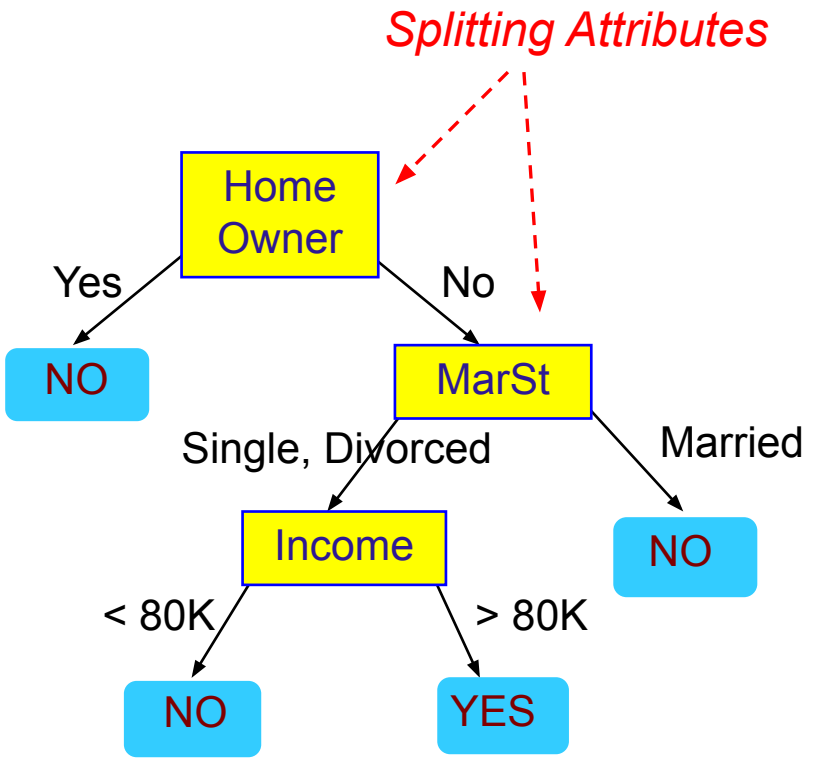

Test Data

| Home Owner | Marital Status | Annual Income | Defaulted Borrower |
|---|---|---|---|
| No | Married | 80K | ? |

# Example of a Decision Tree

| ID | Home Owner | Marital Status | Annual Income | Defaulted Borrower |
|---|---|---|---|---|
| | | categorical | categorical | continuous | class |
| 1 | Yes | Single | 125K | No |
| 2 | No | Married | 100K | No |
| 3 | No | Single | 70K | No |
| 4 | Yes | Married | 120K | No |
| 5 | No | Divorced | 95K | Yes |
| 6 | No | Married | 60K | No |
| 7 | Yes | Divorced | 220K | No |
| 8 | No | Single | 85K | Yes |
| 9 | No | Married | 75K | No |
| 10 | No | Single | 90K | Yes |

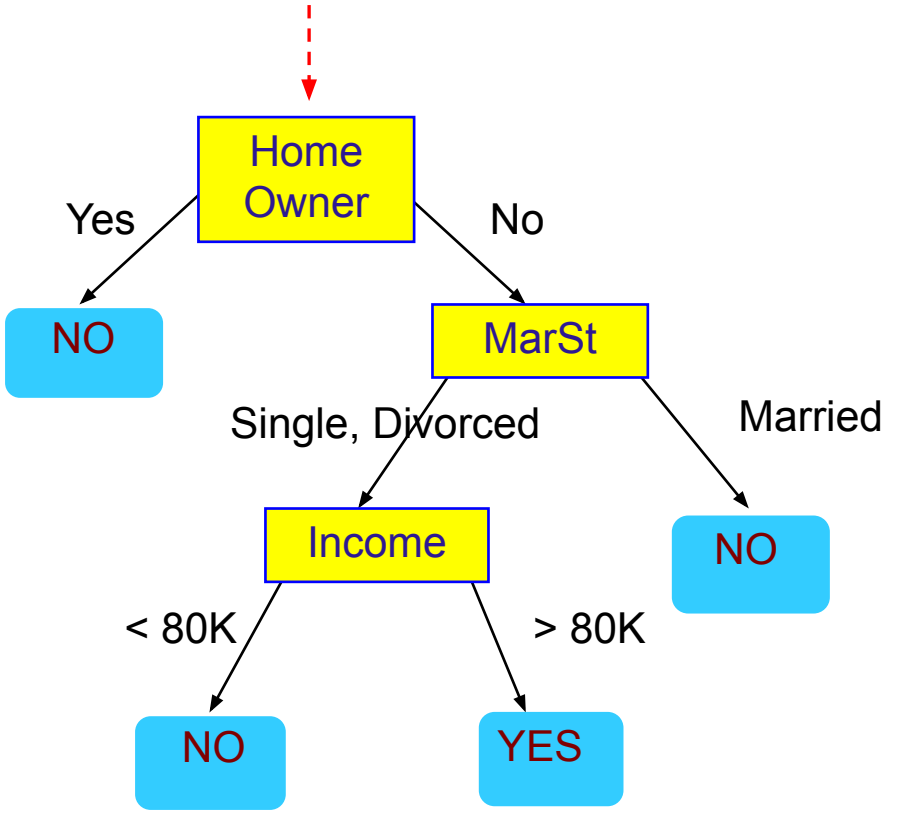
Training Data




Model: Decision Tree

# Apply Model to Test Data

Start from the root of tree.



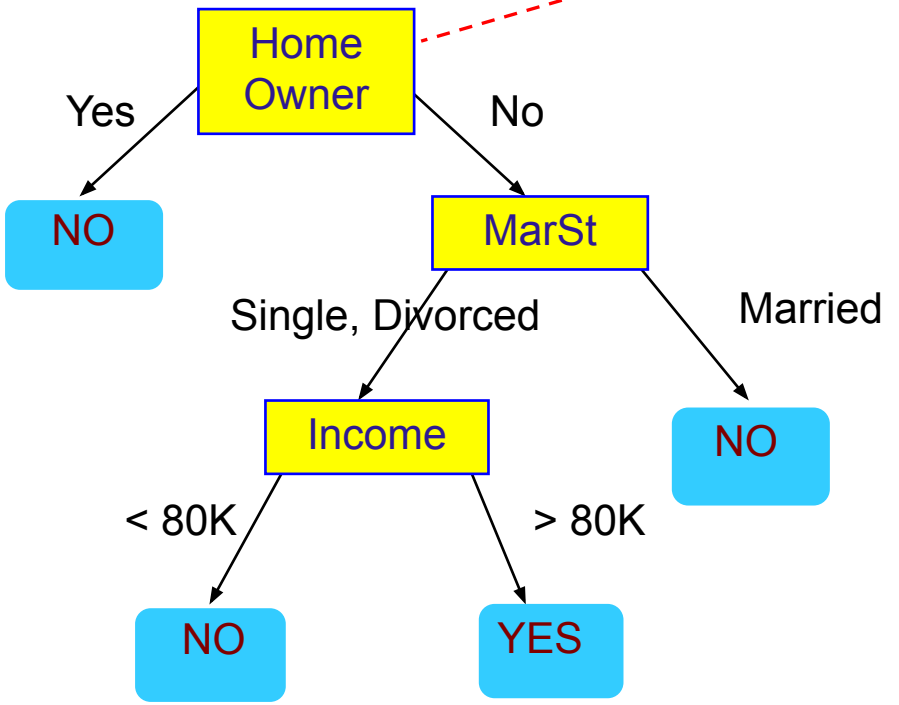

Test Data

| Home Owner | Marital Status | Annual Income | Defaulted Borrower |
|---|---|---|---|
| No | Married | 80K | ? |

# Apply Model to Test Data

Test Data

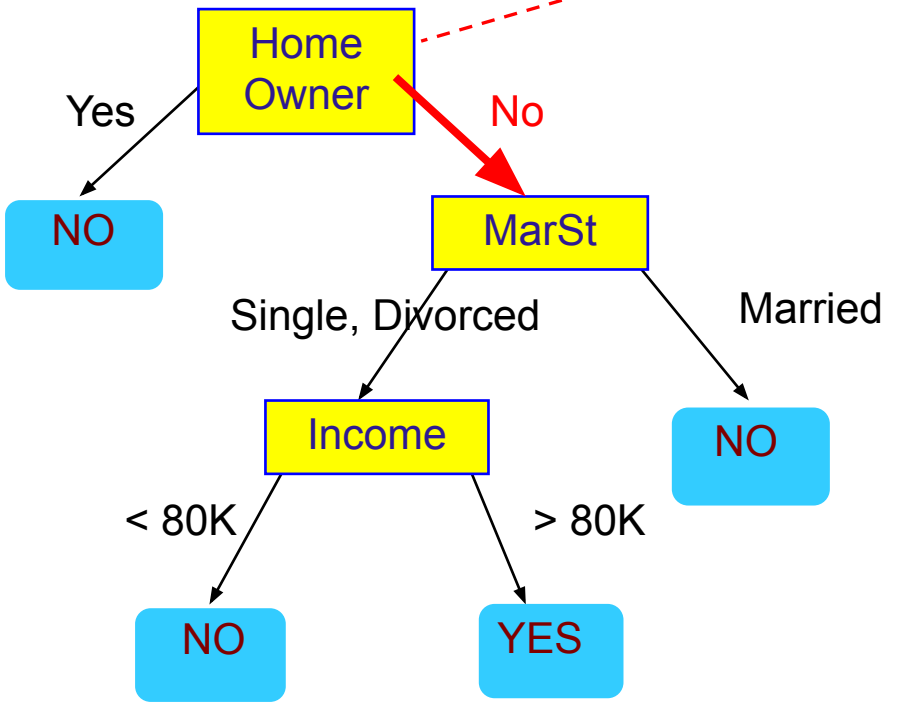
| Home Owner | Marital Status | Annual Income | Defaulted Borrower |
|---|---|---|---|
| No | Married | 80K | ? |



# Apply Model to Test Data

Test Data

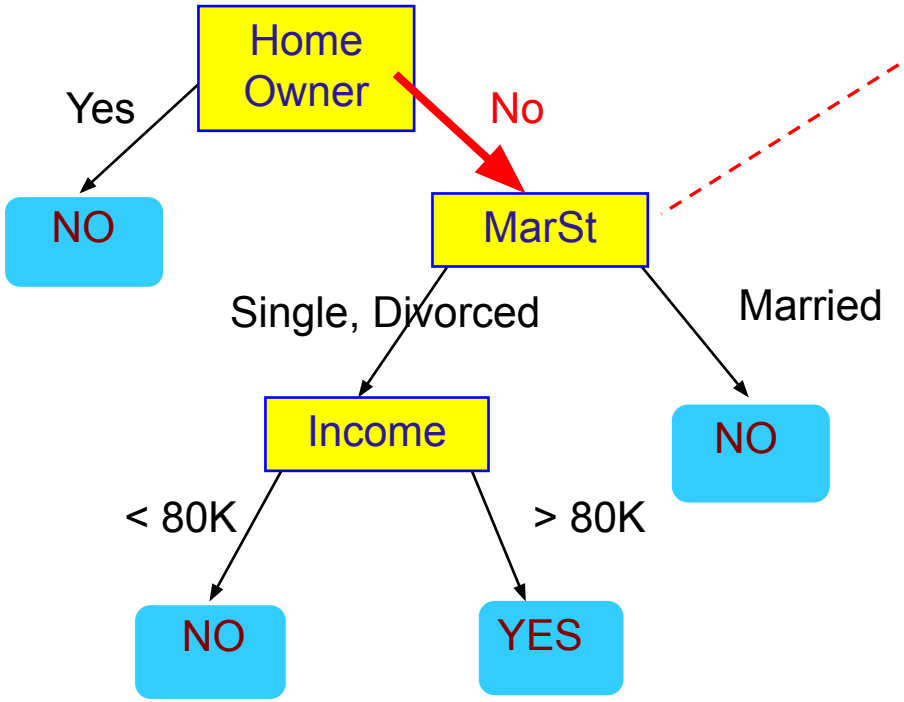
| Home Owner | Marital Status | Annual Income | Defaulted Borrower |
|---|---|---|---|
| No | Married | 80K | ? |



# Apply Model to Test Data

Test Data

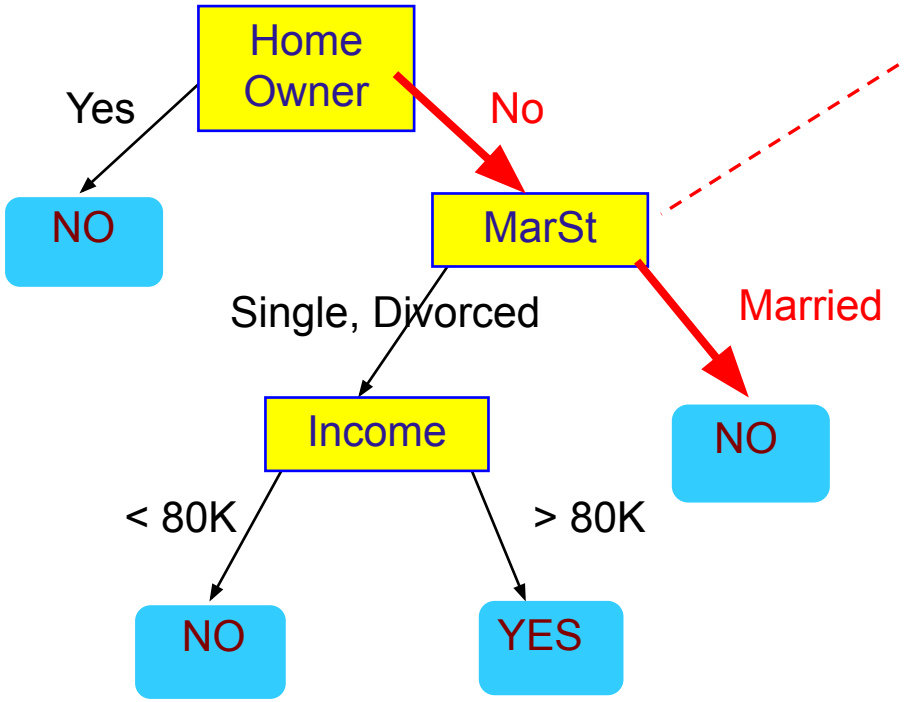
| Home Owner | Marital Status | Annual Income | Defaulted Borrower |
|---|---|---|---|
| No | Married | 80K | ? |



# Apply Model to Test Data

Test Data

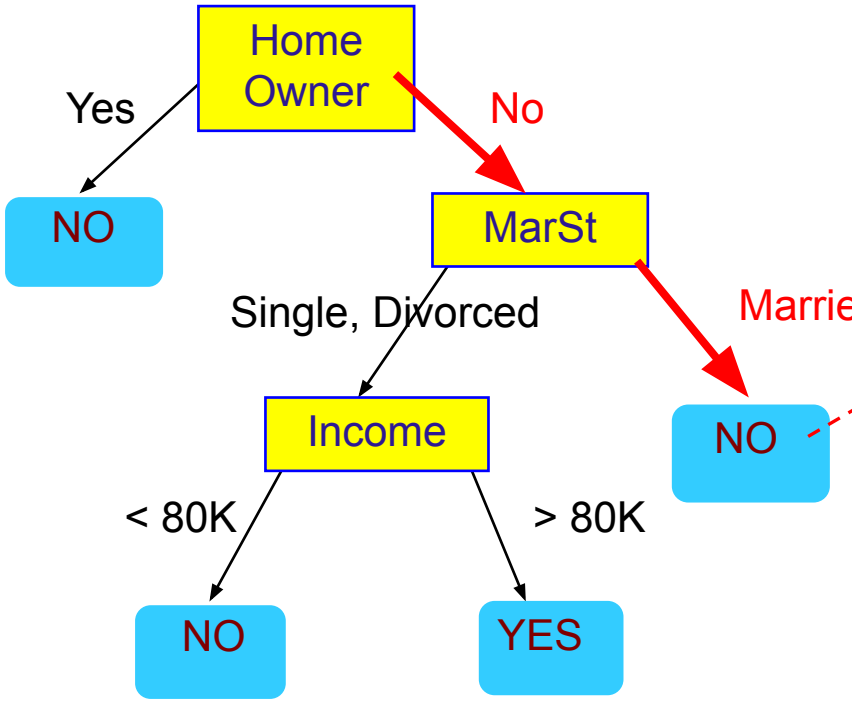
| Home Owner | Marital Status | Annual Income | Defaulted Borrower |
|---|---|---|---|
| No | Married | 80K | ? |



# Apply Model to Test Data

Test Data

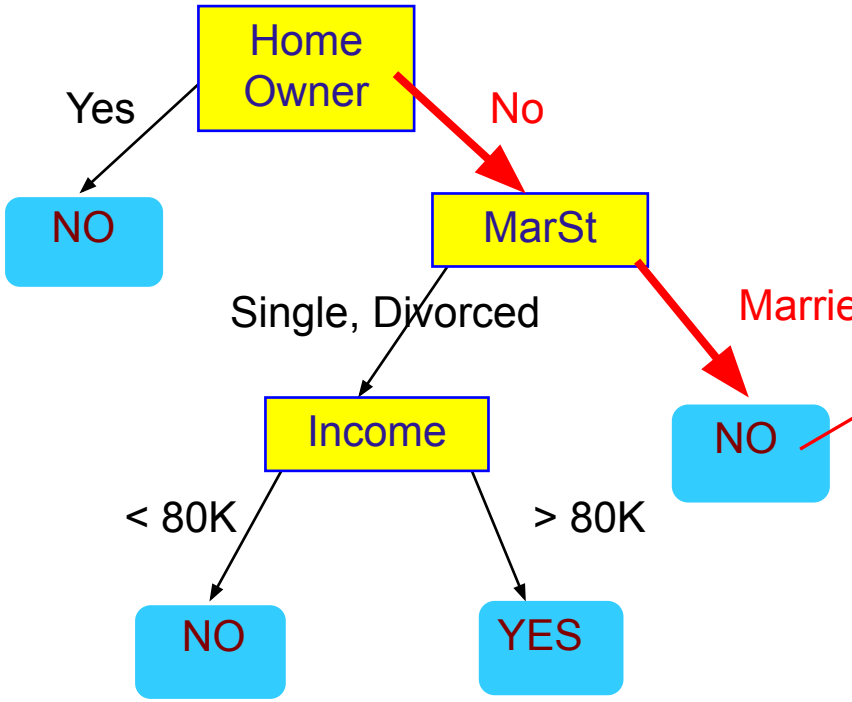
| Home Owner | Marital Status | Annual Income | Defaulted Borrower |
|---|---|---|---|
| No | Married | 80K | ? |



# Apply Model to Test Data

Test Data

| Home Owner | Marital Status | Annual Income | Defaulted Borrower |
|---|---|---|---|
| No | Married | 80K | ? |

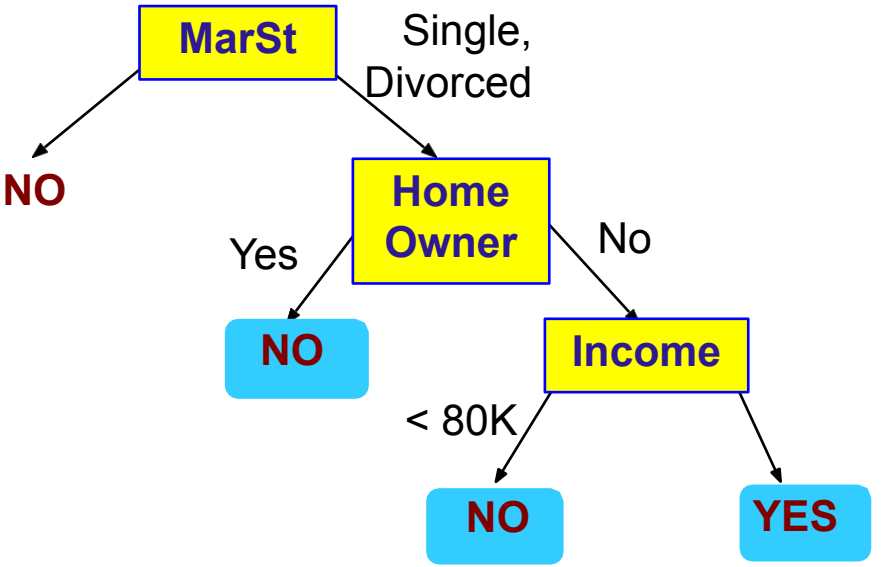



Assign Defaulted to "No"

# Another example of Decision Tree

RV College of Engineering

| ID | Home Owner (categorical) | Marital Status (categorical) | Annual Income (continuous) | Defaulted Borrower (class) |
|---|---|---|---|---|
| 1 | Yes | Single | 125K | No |
| 2 | No | Married | 100K | No |
| 3 | No | Single | 70K | No |
| 4 | Yes | Married | 120K | No |
| 5 | No | Divorced | 95K | Yes |
| 6 | No | Married | 60K | No |
| 7 | Yes | Divorced | 220K | No |
| 8 | No | Single | 85K | Yes |
| 9 | No | Married | 75K | No |
| 10 | No | Single | 90K | Yes |




**There could be more than one tree that fits the same data!**

# 1.Basic Issues in DTLearning

## 1.1 Concept learning: An example

Given the data:

| Day | Outlook | Temperature | Humidity | Wind | EnjoyTennis |
|---|---|---|---|---|---|
| D1 | Sunny | Hot | High | Weak | No |
| D2 | Sunny | Hot | High | Strong | No |
| D3 | Overcast | Hot | High | Weak | Yes |
| D4 | Rain | Mild | High | Weak | Yes |
| D5 | Rain | Cool | Normal | Weak | Yes |
| D6 | Rain | Cool | Normal | Strong | No |
| D7 | Overcast | Cool | Normal | Strong | Yes |
| D8 | Sunny | Mild | High | Weak | No |
| D9 | Sunny | Cool | Normal | Weak | Yes |
| D10 | Rain | Mild | Normal | Weak | Yes |
| D11 | Sunny | Mild | Normal | Strong | Yes |
| D12 | Overcast | Mild | High | Strong | Yes |
| D13 | Overcast | Hot | Normal | Weak | Yes |
| D14 | Rain | Mild | High | Strong | No |

predict the value of EnjoyTennis for

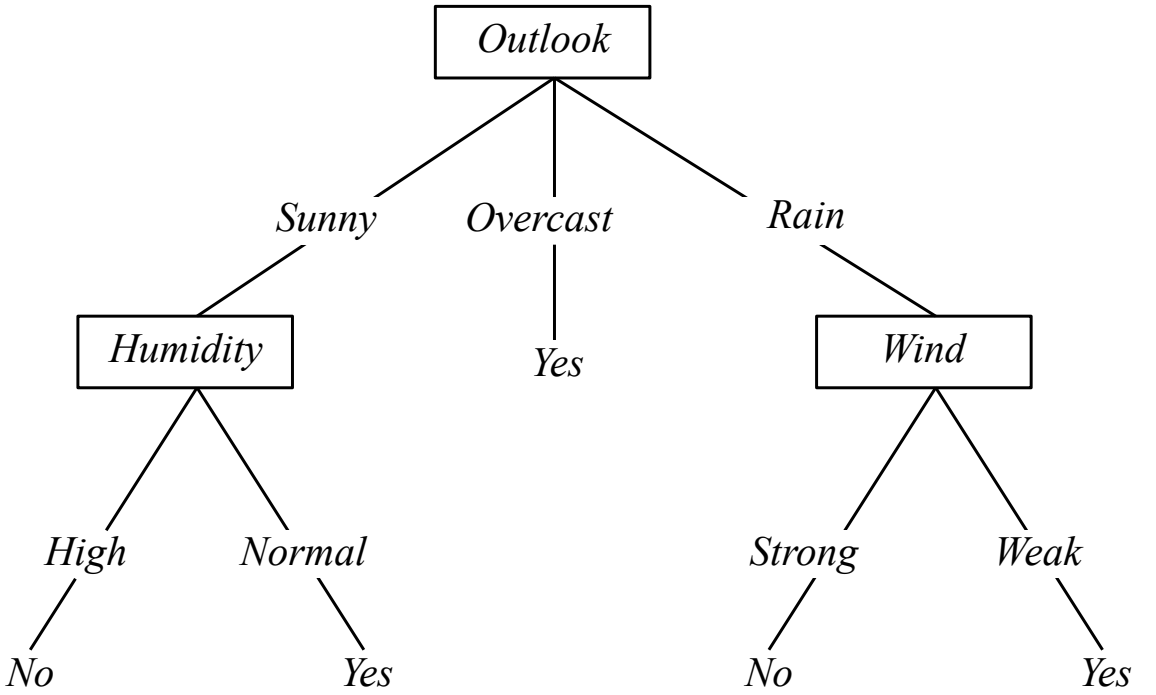
$\langle Outlook = sunny, Temp = cool, Humidity = high, Wind = strong \rangle$

# 1.2. Decision tree representation

- Each internal node tests an attribute
- Each branch corresponds to attribute value
- Each leaf node assigns a classification

Example:

Decision Tree for *EnjoyTennis*



RV College of Engineering®

# Another example: A Tree to Predict C-SectionRisk

Learned from medical records of 1000 women

Negative examples are C-sections

[833+,167-] .83+ .17-
Fetal_Presentation= 1: [822+,116-] .88+.12-
| Previous_Csection = 0: [767+,81-] .90+.10-
| | Primiparous = 0: [399+,13-] .97+.03-
| | Primiparous = 1: [368+,68-] .84+.16-
| | | Fetal_Distress = 0: [334+,47-] .88+ .12-
| | | | Birth_Weight < 3349: [201+,10.6-] .95+ .05-
| | | | Birth_Weight >= 3349: [133+,36.4-] .78+.22-
| | | Fetal_Distress = 1: [34+,21-] .62+.38-
| Previous_Csection = 1: [55+,35-] .61+.39-
Fetal_Presentation= 2: [3+,29-].11+ .89-
Fetal_Presentation= 3: [8+,22-].27+ .73-

RV College of Engineering

# When to Consider Decision Trees

- Instances are described by attribute–value pairs- each example is represented by features (attributes) and their values.
- Target function is discrete valued- The output to be predicted is categorical (e.g., yes/no, class labels).
- Disjunctive hypothesis may be required- Sometimes multiple conditions must be combined with OR to describe the target concept.
- Possibly noisy training data- Data may contain errors or inconsistencies that affect learning.
- Training data may contain missing attribute values- Some features may be incomplete, requiring the learner to handle gaps.

# 1.3.Top-Down Induction of Decision Trees

ID3 algorithm outline [Ross Quinlan, 1979, 1986]

START

create the root *node*;
assign all examples to root;

Main loop:

1. *A* ⟶ the "best" decision attribute for next *node*;
2. for each value of *A*, create a new descendant of *node*;
3. sort training examples to leaf nodes;
4. if training examples perfectly classified, then STOP; else iterate over new leaf nodes

# ID3 Algorithm: basic version

ID3(*Examples, Target_attribute, Attributes*)

- create a *Root* node for the tree; assign all *Examples* to *Root*;
- if all *Examples* are positive, return the single-node tree *Root*, with label=+;
- if all *Examples* are negative, return the single-node tree *Root*, with label=−;
- if *Attributes* is empty, return the single-node tree *Root*,
  with label = the most common value of *Target_attribute* in *Examples*;
- otherwise // Main loop:
  - $A \rightarrow$ the attribute from *Attributes* that best* classifies *Examples*; the decision attribute for *Root* $\rightarrow A$;
  - for each possible value $v_i$ of $A$
    - add a new tree branch below *Root*, corresponding to the test $A = v_i$; let $Examples_{vi}$ be the subset of *Examples* that have the value $v_i$ for $A$; if $Examples_{vi}$ is empty
      - below this new branch add a leaf node with label = the most common value of *Target attribute* in *Examples*;
    - else
      - below this new branch add the subtree ID3($Examples_{vi}$, *Target attribute, Attributes*\{$A$});
- return *Root*;

* The *best* attribute is the one with the highest *information gain*.

# Entropy

- Let $S$ be a sample of training examples

  $p_{\oplus}$ is the proportion of positive examples in $S$ $p_{\ominus}$ is the proportion of negative examples in $S$

- Entropy measures the impurity of $S$

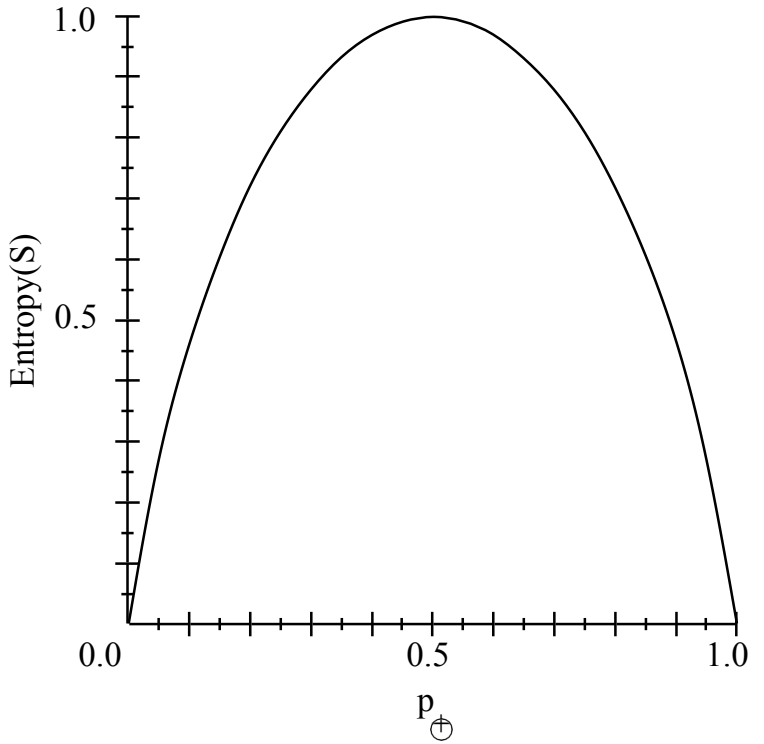
- Information theory:

  *Entropy*($S$) = expected number of bits needed to encode $\oplus$ or $\ominus$ for a randomly drawn member of $S$ (under the optimal, shortest- length code)

  The optimal length code for a message having the probability $p$ is $-\log_2 p$ bits. So:

$$Entropy(S) = p_{\oplus}(-\log_2 p_{\oplus}) + p_{\ominus}(-\log_2 p_{\ominus}) = -p_{\oplus}\log_2 p_{\oplus} - p_{\ominus}\log_2 p_{\ominus}$$

# Entropy




$$Entropy(S) = p_{\oplus} \log_2 \frac{1}{p_{\oplus}} + p_{\ominus} \log_2 \frac{1}{p_{\ominus}}$$

Note: By convention, $0 \cdot \log_2 0 = 0$.

# Statistical measures in DTlearning: Entropy, and Information Gain

## Information gain:

the expected reduction of the entropy of the instance set $S$ due to sorting on the attribute $A$

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Where:

- $S$ is the original dataset
- $A$ is the attribute
- $S_v$ is the subset of $S$ where attribute $A = v$

# 1.Basic Issues in DTLearning

## 1.1 Concept learning: An example

Given the data:

| Day | Outlook | Temperature | Humidity | Wind | EnjoyTennis |
|---|---|---|---|---|---|
| D1 | Sunny | Hot | High | Weak | No |
| D2 | Sunny | Hot | High | Strong | No |
| D3 | Overcast | Hot | High | Weak | Yes |
| D4 | Rain | Mild | High | Weak | Yes |
| D5 | Rain | Cool | Normal | Weak | Yes |
| D6 | Rain | Cool | Normal | Strong | No |
| D7 | Overcast | Cool | Normal | Strong | Yes |
| D8 | Sunny | Mild | High | Weak | No |
| D9 | Sunny | Cool | Normal | Weak | Yes |
| D10 | Rain | Mild | Normal | Weak | Yes |
| D11 | Sunny | Mild | Normal | Strong | Yes |
| D12 | Overcast | Mild | High | Strong | Yes |
| D13 | Overcast | Hot | Normal | Weak | Yes |
| D14 | Rain | Mild | High | Strong | No |

predict the value of EnjoyTennis for

$\langle Outlook = sunny, Temp = cool, Humidity = high, Wind = strong \rangle$

RV College of Engineering®

# A partially learnedtree

## Attribute: Outlook

$$Values\,(Outlook) = Sunny, Overcast, Rain$$

$$S = [9+, 5-] \qquad Entropy(S) = -\frac{9}{14} log_2 \frac{9}{14} - \frac{5}{14} log_2 \frac{5}{14} = 0.94$$

$$S_{Sunny} \leftarrow [2+, 3-] \qquad Entropy(S_{Sunny}) = -\frac{2}{5} log_2 \frac{2}{5} - \frac{3}{5} log_2 \frac{3}{5} = 0.971$$

$$S_{Overcast} \leftarrow [4+, 0-] \qquad Entropy(S_{Overcast}) = -\frac{4}{4} log_2 \frac{4}{4} - \frac{0}{4} log_2 \frac{0}{4} = 0$$

$$S_{Rain} \leftarrow [3+, 2-] \qquad Entropy(S_{Rain}) = -\frac{3}{5} log_2 \frac{3}{5} - \frac{2}{5} log_2 \frac{2}{5} = 0.971$$

$$Gain\,(S, Outlook) = Entropy(S) - \sum_{v \in \{Sunny, Overcast, Rain\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Outlook)$$

$$= Entropy(S) - \frac{5}{14} Entropy(S_{Sunny}) - \frac{4}{14} Entropy(S_{Overcast})$$

$$- \frac{5}{14} Entropy(S_{Rain})$$

$$Gain(S, Outlook) = 0.94 - \frac{5}{14} 0.971 - \frac{4}{14}\, 0 - \frac{5}{14} 0.971 = 0.2464$$

RV College of Engineering®

# A partially learnedtree

## Attribute: Temp

$$Values\ (Temp) = Hot, Mild, Cool$$

$$S = [9+, 5-] \qquad Entropy(S) = -\frac{9}{14} log_2 \frac{9}{14} - \frac{5}{14} log_2 \frac{5}{14} = 0.94$$

$$S_{Hot} \leftarrow [2+, 2-] \qquad Entropy(S_{Hot}) = -\frac{2}{4} log_2 \frac{2}{4} - \frac{2}{4} log_2 \frac{2}{4} = 1.0$$

$$S_{Mild} \leftarrow [4+, 2-] \qquad Entropy(S_{Mild}) = -\frac{4}{6} log_2 \frac{4}{6} - \frac{2}{6} log_2 \frac{2}{6} = 0.9183$$

$$S_{Cool} \leftarrow [3+, 1-] \qquad Entropy(S_{Cool}) = -\frac{3}{4} log_2 \frac{3}{4} - \frac{1}{4} log_2 \frac{1}{4} = 0.8113$$

$$Gain\ (S, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Temp)$$

$$= Entropy(S) - \frac{4}{14} Entropy(S_{Hot}) - \frac{6}{14} Entropy(S_{Mild})$$

$$- \frac{4}{14} Entropy(S_{Cool})$$

RV College of Engineering®

# A partially learnedtree

## Attribute: Humidity

$Values\,(Humidity) = High, Normal$

$S = [9+, 5-]$ $\quad Entropy(S) = -\frac{9}{14} log_2 \frac{9}{14} - \frac{5}{14} log_2 \frac{5}{14} = 0.94$

$S_{High} \leftarrow [3+, 4-]$ $\quad Entropy(S_{High}) = -\frac{3}{7} log_2 \frac{3}{7} - \frac{4}{7} log_2 \frac{4}{7} = 0.9852$

$S_{Normal} \leftarrow [6+, 1-]$ $\quad Entropy(S_{Normal}) = -\frac{6}{7} log_2 \frac{6}{7} - \frac{1}{7} log_2 \frac{1}{7} = 0.5916$

$$Gain\,(S, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Humidity)$$

$$= Entropy(S) - \frac{7}{14} Entropy(S_{High}) - \frac{7}{14} Entropy(S_{Normal})$$

RV College of Engineering®

# A partially learnedtree

## Attribute: Wind

$Values\,(Wind) = Strong, Weak$

$S = [9+, 5-]$ $\quad Entropy(S) = -\frac{9}{14} log_2 \frac{9}{14} - \frac{5}{14} log_2 \frac{5}{14} = 0.94$

$S_{Strong} \leftarrow [3+, 3-]$ $\quad Entropy(S_{Strong}) = 1.0$

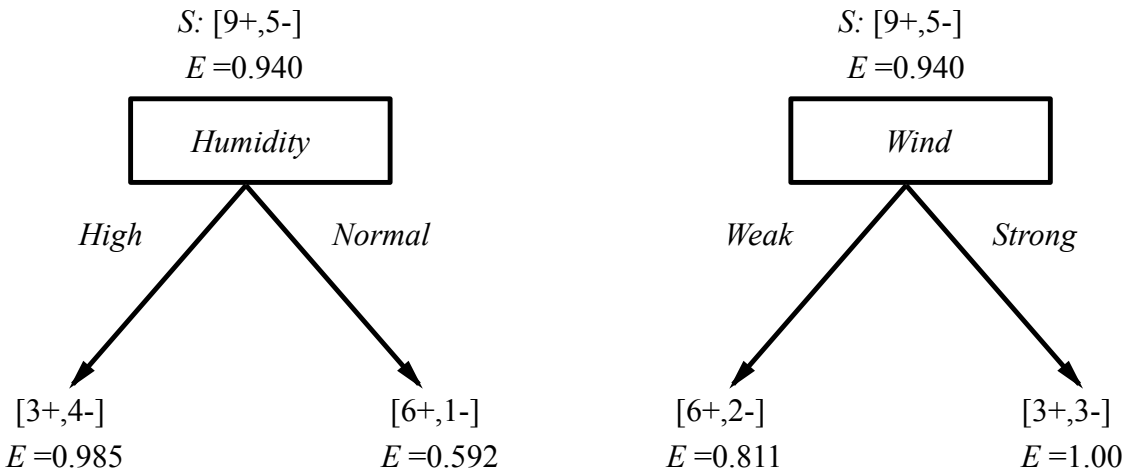
$S_{Weak} \leftarrow [6+, 2-]$ $\quad Entropy(S_{Weak}) = -\frac{6}{8} log_2 \frac{6}{8} - \frac{2}{8} log_2 \frac{2}{8} = 0.8113$

$$Gain\,(S, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Wind) = Entropy(S) - \frac{6}{14} Entropy(S_{Strong}) - \frac{8}{14} Entropy(S_{Weak})$$

# Selecting the Root Attribute

**Which attribute is the best classifier?**



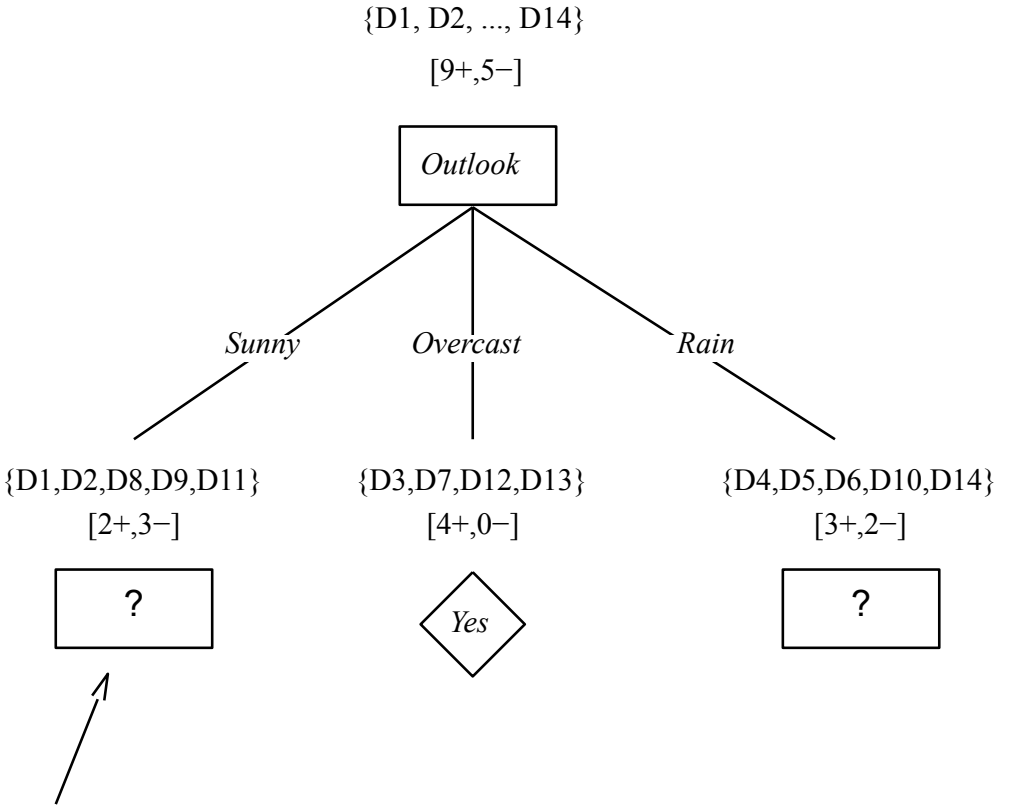

*Gain (S, Humidity )*
= .940 - (7/14).985 - (7/14).592
= .151

*Gain (S, Wind)*
= .940 - (8/14).811 - (6/14)1.0
= .048

Similarly,

*Gain*(*S*, *Outlook* ) = 0.246
*Gain*(*S*, *Temperature*) = 0.029

# A partially learnedtree




*Which attribute should be tested here?*

$S_{sunny}$ = {D1,D2,D8,D9,D11}

*Gain* ($S_{sunny}$, *Humidity*) = .970 − (3/5) 0.0 − (2/5) 0.0 = .970

*Gain* ($S_{sunny}$, *Temperature*) = .970 − (2/5) 0.0 − (2/5) 1.0 − (1/5) 0.0 = .570

*Gain* ($S_{sunny}$, *Wind*) = .970 − (2/5) 1.0 − (3/5) .918 = .019

RV College of Engineering®

# A partially learnedtree

| Day | Temp | Humidity | Wind | Play Tennis |
|---|---|---|---|---|
| D1 | Hot | High | Weak | No |
| D2 | Hot | High | Strong | No |
| D8 | Mild | High | Weak | No |
| D9 | Cool | Normal | Weak | Yes |
| D11 | Mild | Normal | Strong | Yes |

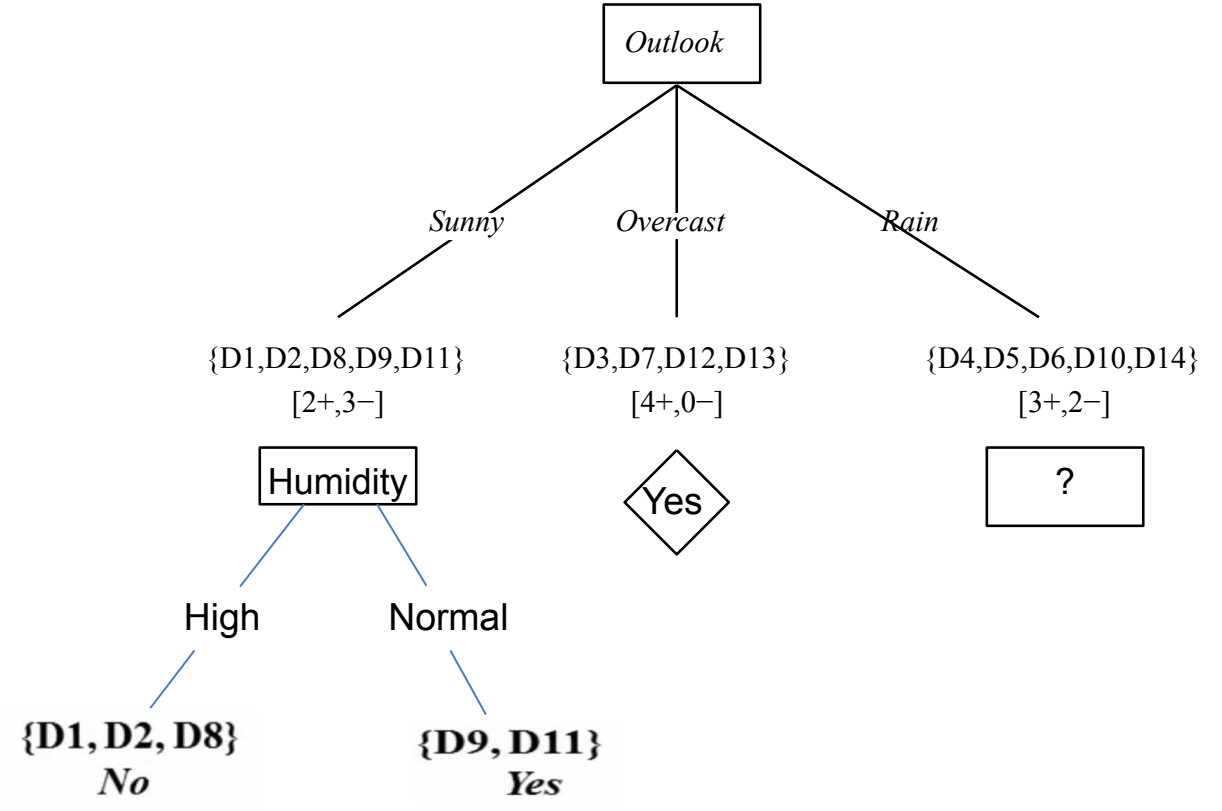
$$Gain(S_{sunny}, Temp) = 0.570$$

$$Gain(S_{sunny}, Humidity) = 0.97$$

$$Gain(S_{sunny}, Wind) = 0.0192$$

RV College of Engineering

# A partially learnedtree



RV College of Engineering®

# A partially learnedtree

| Day | Temp | Humidity | Wind | Play Tennis |
|---|---|---|---|---|
| D4 | Mild | High | Weak | Yes |
| D5 | Cool | Normal | Weak | Yes |
| D6 | Cool | Normal | Strong | No |
| Dl0 | Mild | Normal | Weak | Yes |
| Dl4 | Mild | High | Strong | No |

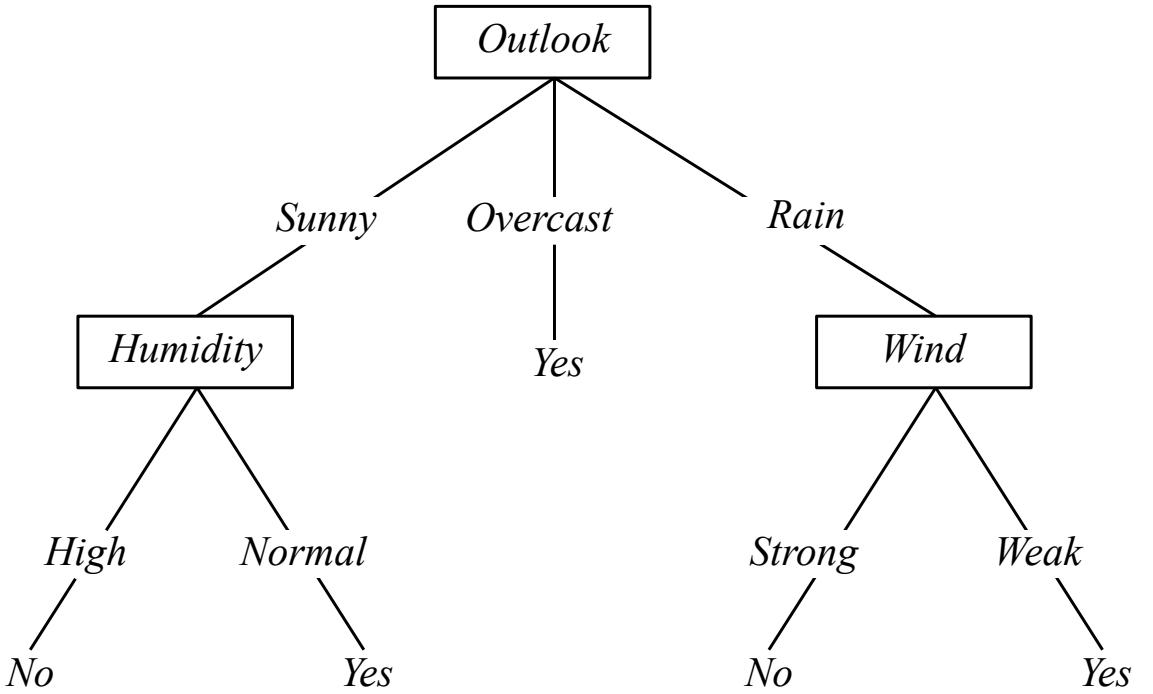
$$Gain(S_{Rain}, Temp) = 0.0192$$

$$Gain(S_{Rain}, Humidity) = 0.0192$$

$$Gain(S_{Rain}, Wind) = 0.97$$

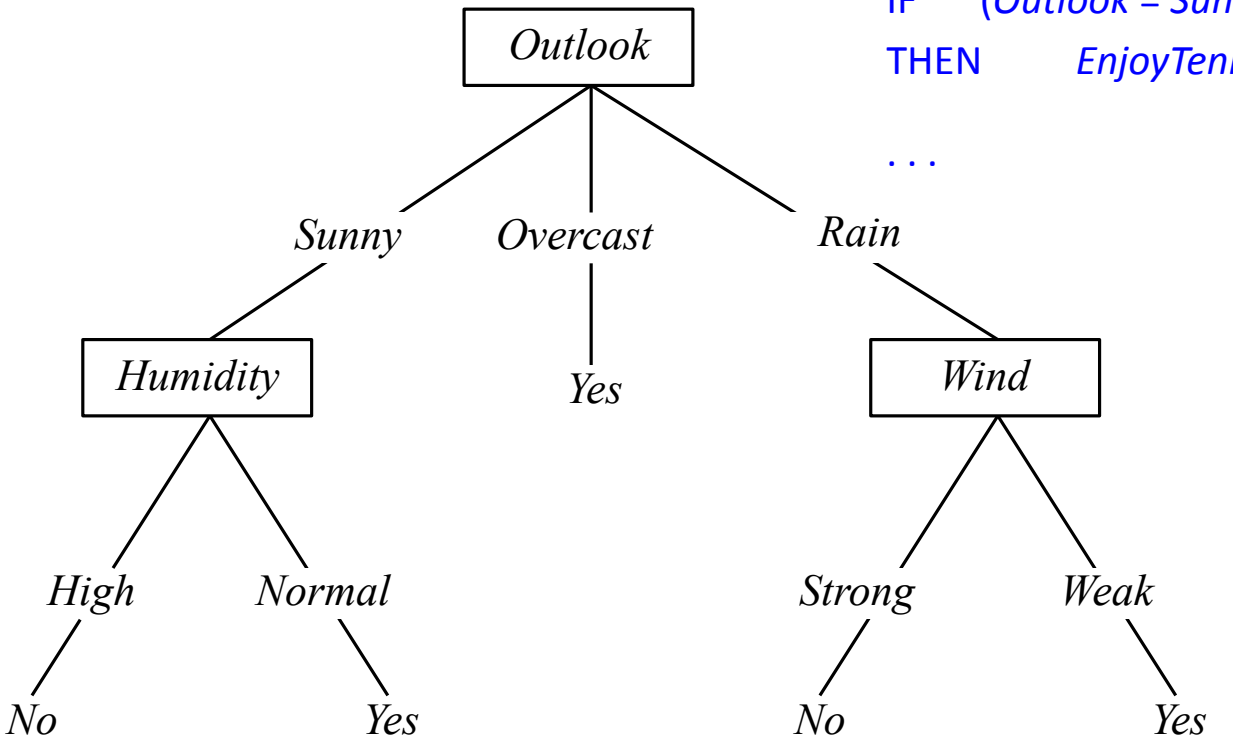
RV College of Engineering®

# Final Tree



RV College of Engineering

# Converting A Tree to Rules




IF (*Outlook = Sunny*) $\wedge$ (*Humidity = High*)
THEN *EnjoyTennis = No*

IF (*Outlook = Sunny*) $\wedge$ (*Humidity = Normal*)
THEN *EnjoyTennis = Y es*

. . .

# 1.4 Hypothesis SpaceSearch by ID3



- Hypothesis space is complete!

  *The target function surely is in there...*

- Outputs a single hypothesis

  *Which one?*

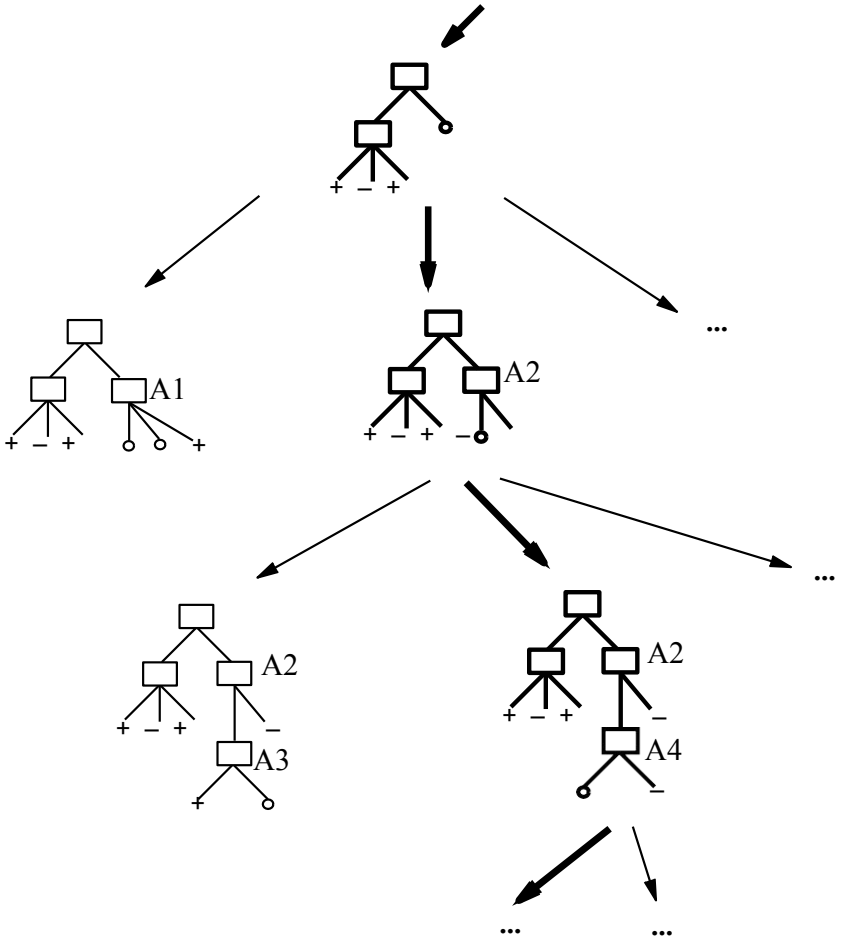
- *Approximate inductive bias of ID3: Shorter trees are preferred over larger trees.*

- Statisically-based search choices

  *Robust to noisy data...*

- No back-tracking

  *Local minima...*



# Occam's Razor

- Prefer the simplest hypothesis that fits the data.

Why prefer short hypotheses?

Argument in favor:

- Fewer short hypotheses than long hypsotheses

→ a short hypothesis that fits data unlikely to be coincidence

→ a long hypothesis that fits data might be coincidence

**What Makes It "Simple"?**

"Simple" refers to:

**Fewer nodes or branches** in the tree.

**Shallow depth** (not too many levels).

**Minimal assumptions**—it doesn't overfit or memorize the data.

# 1.5 Complexity of decision tree induction

from "Data mining. Practical machine learning tools and techniques" Witten et al, 3rd ed., 2011, pp. 199-200

- Input: *d* attributes, and *m* training instances
- Simplifying assumptions:

  (A1): the depth of the ID3 tree is $O(\log m)$,

  (i.e. it remains "bushy" and doesn't degenerate into long, stringy branches);

  (A2): [most] instances differ from each other;

  (A2'): the *d* attributes provide enough tests to allow the instances to be differentiated.

- Time complexity: $O(d\, m \log m)$.

RV College of Engineering®

# Characteristics of Decision Tree Classifiers



- **Flexible & Nonparametric**: No assumptions about data distribution; handles both categorical and continuous data.
- **Multiclass Support**: Directly handles multiple classes—no need for binary decomposition.
- **Interpretable**: Easy to visualize and explain, especially when shallow.
- **Efficient**: Fast to train and test; uses greedy recursive partitioning.
- **Expressive**: Can represent complex discrete functions compactly.
- **Handles Missing Values**: Uses probabilistic, surrogate, or separate-class strategies.
- **Sensitive to Attribute Interactions**: May miss useful combinations due to greedy splits.

RV College of Engineering®

# Decision Tree Induction

- Many Algorithms:
  - Hunt's Algorithm (one of the earliest)
  - CART
  - ID3, C4.5
  - SLIQ,SPRINT

RV College of Engineering

# Design Issues of Decision Tree Induction

- How should training records be split?
  - Method for expressing test condition
    - depending on attribute types
  - Measure for evaluating the goodness of a test condition

- How should the splitting procedure stop?
  - Stop splitting if all the records belong to the same class or have identical attribute values
  - Early termination

RV College of Engineering

# Design Issues of Decision Tree Induction

**1. Overfitting**

Trees grown too deep may fit noise or small sample quirks.

Solutions: **early stopping** or **post-pruning** (e.g., reduced-error pruning, rule post-pruning).

**2. Continuous Attributes**

ID3 originally handles only discrete values.

Solution: **convert continuous attributes into thresholds/intervals** (e.g., Temperature < 60).

**3. Attribute Selection Measures**

Information Gain favors attributes with many values (like Date).

Solutions: **Gain Ratio** or **distance-based measures** to avoid bias.

RV College of Engineering

# Design Issues of Decision Tree Induction

**4. Missing Attribute Values**

Training data may lack values for some features.

Solutions:

Fill with most common value.

Use probabilities to distribute fractional examples (used in C4.5).

**5. Attributes with Differing Costs**

Some attributes are expensive (e.g., medical tests).

Solution: **cost-sensitive measures** that prefer low-cost attributes unless high-cost ones are necessary.

RV College of Engineering

# Advantages & Disadvantages of Decision Trees

- Advantages:
  - Relatively inexpensive to construct
  - Extremely fast at classifying unknown records
  - Easy to interpret for small-sized trees
  - Robust to noise (especially when methods to avoid overfitting are employed)
  - Can easily handle redundant attributes
  - Can easily handle irrelevant attributes (unless the attributes are interacting)
- Disadvantages: .
  - Due to the greedy nature of splitting criterion, interacting attributes (that can distinguish between classes together but not individually) may be passed over in favor of other attributed that are less discriminating.
  - Each decision boundary involves only a single attribute

RV College of Engineering®

# Credits

1. Data mining. Practical machine learning tools and techniques.
2. Adaptation of slides by Tom Mitchell for Machine Learning, CMU.